PENTECOST UNIVERSITY

Faculty of Information Technology and Computing

BSc Information Technology

HEART DISEASE SPECIES CLASSIFICATION USING MACHINE LEARNING

A Machine Learning Project Report Submitted in Partial Fulfilment of the Requirements for the BSc Information Technology Programme

Student Name: Philip Antwi

Student ID: .....

Supervisor: .....

Submission Date: .....

Abstract

Heart disease remains one of the leading causes of mortality worldwide. Early and accurate prediction of heart disease risk is critical for timely medical intervention and improved patient outcomes. This project presents the design, development, and deployment of a machine learning system for classifying the presence or absence of heart disease in patients based on ten medical and lifestyle features. A synthetic hospital dataset comprising 500 patient records was used to train and evaluate two classification algorithms: Logistic Regression and Random Forest. Logistic Regression was selected as the best-performing model with a test accuracy of 56.00%. The trained model was integrated into a Python serverless API and deployed on the Vercel cloud platform, with a responsive HTML/CSS/JavaScript front-end that enables real-time patient risk assessment. The system architecture, preprocessing pipeline, model evaluation metrics, and deployment workflow are discussed in detail. The project demonstrates the practical application of machine learning in healthcare prediction tasks and highlights key challenges encountered during development.

Table of Contents

1. Introduction 4
2. Problem Statement 4
3. Project Aim and Objectives 5
4. Significance of the Study 5
5. Literature Review 5
6. Dataset Description and Exploration 6
7. Data Preprocessing and Preparation 7
8. Logistic Regression Algorithm 7
9. Model Training and Evaluation 8
10. Results 9
11. System Design and Implementation 10
12. Website Deployment and Testing 11
13. Challenges and Limitations 12
14. Personal Reflection 12
15. Conclusion and Recommendations 13
16. References 13
17. Appendices 14

List of Figures

Figure 1 – Class Distribution of Heart Disease Dataset 6
Figure 2 – Confusion Matrix – Logistic Regression 9
Figure 3 – ROC Curve – Logistic Regression 9
Figure 4 – Feature Importance Chart 10
Figure 5 – System Architecture Diagram 11

1. Introduction

Cardiovascular disease is the leading cause of death globally, accounting for approximately 17.9 million deaths per year according to the World Health Organization [1]. The ability to predict heart disease risk at an early stage can significantly improve patient survival rates through timely medical intervention. Machine learning has emerged as a powerful tool for analysing complex medical datasets and generating predictive insights that assist healthcare practitioners in clinical decision-making [2].

This project develops a Heart Disease Classification System using Python and the scikit-learn machine learning library. The system is trained on a synthetic hospital dataset of 500 patient records and deployed as a web application using Vercel, allowing users to input patient information and receive an instant prediction of heart disease risk. The project integrates a static HTML/CSS/JavaScript front-end with a Python serverless API, forming a complete, production-ready machine learning deployment pipeline.

2. Problem Statement

Despite significant advances in medical technology, heart disease is frequently diagnosed at advanced stages when treatment options become limited and costs rise substantially. Many rural and resource-limited healthcare settings in Ghana and across Africa lack access to specialised diagnostic tools and cardiologists [3]. There is therefore a pressing need for affordable, accessible, and data-driven screening tools that can assist primary healthcare workers in identifying patients at high risk of heart disease early.

This project addresses that gap by developing a machine learning classification model that predicts whether a patient is likely to have heart disease based on ten commonly available medical and lifestyle indicators. The model is deployed as a freely accessible web application that requires no specialised hardware or software installations.

3. Project Aim and Objectives

The primary aim of this project is to design, train, evaluate, and deploy a machine learning model capable of accurately classifying the presence or absence of heart disease in patients based on readily available medical and lifestyle data.

The specific objectives are:

- To explore and preprocess the synthetic hospital patient dataset.
- To implement and compare Logistic Regression and Random Forest classifiers.

- To evaluate model performance using accuracy, confusion matrix, ROC-AUC, and classification report.
- To develop a responsive web-based front-end for user interaction.
- To expose the trained model as a Vercel serverless Python API.
- To deploy the complete application on Vercel for public access.

4. Significance of the Study

This project demonstrates the practical integration of machine learning into a deployable healthcare application. It is significant for several reasons. First, it provides a low-cost, web-accessible screening tool that can complement clinical diagnosis in resource-limited settings. Second, it bridges the gap between academic machine learning research and real-world software deployment using modern cloud platforms such as Vercel [4]. Third, it serves as a reference implementation for future medical machine learning projects at Pentecost University and other academic institutions.

5. Literature Review

Machine learning has been widely applied to medical diagnosis and prognosis tasks. Mohan et al. [5] demonstrated that hybrid random forest and linear models could achieve accuracy rates above 88% on the UCI Heart Disease dataset, establishing a strong baseline for binary heart disease classification. Similarly, Nashif et al. [6] applied Support Vector Machines (SVM) and Logistic Regression to electrocardiogram and clinical features, finding that Logistic Regression offered competitive performance with high interpretability — a critical factor in clinical contexts.

Pedregosa et al. [7] introduced scikit-learn, the Python library used in this project, as a comprehensive and accessible framework for machine learning. Its standardised API for model training, evaluation, and serialisation makes it an ideal choice for academic and production deployments. The StandardScaler used in this project is a well-established technique for normalising features prior to model training, as recommended by the scikit-learn documentation.

Regarding deployment, Vercel [4] has emerged as a popular cloud platform for deploying serverless functions and static websites. Its support for Python runtime functions through the @vercel/python builder makes it particularly suitable for hosting machine learning APIs without requiring dedicated server infrastructure. This approach aligns with the growing trend of serverless machine learning inference described by Bhatt et al. [8].

6. Dataset Description and Exploration

The dataset used in this project is a synthetic hospital patient dataset stored in an Excel file (hospital_synthetic_dataset.xlsx). It contains 500 patient records across 11 columns: 10 feature columns and 1 binary target column (heart_disease: 0 = No, 1 = Yes).

The features are as follows:

Feature	Description	Type	Range/Values
age	Patient age in years	Numerical	30 – 79
sex	Patient sex (Male/Female)	Categorical	Male, Female
blood_pressure	Systolic blood pressure in mmHg	Numerical	90 – 179
cholesterol	Serum cholesterol in mg/dL	Numerical	Numerical
bmi	Body Mass Index	Numerical	Continuous
smoker	Whether the patient smokes (Yes/No)	Categorical	Yes, No
diabetes	Whether the patient has diabetes	Categorical	Yes, No
physical_activity	Days per week of physical activity	Numerical	0 – 7
family_history	Family history of heart disease	Categorical	Yes, No
stress_level	Self-reported stress level (1–10)	Numerical	1 – 9

An exploratory analysis revealed that the dataset is balanced, with 234 patients (46.8%) having no heart disease and 266 patients (53.2%) having heart disease. No missing values were found in any column. The class distribution is illustrated in Figure 1.

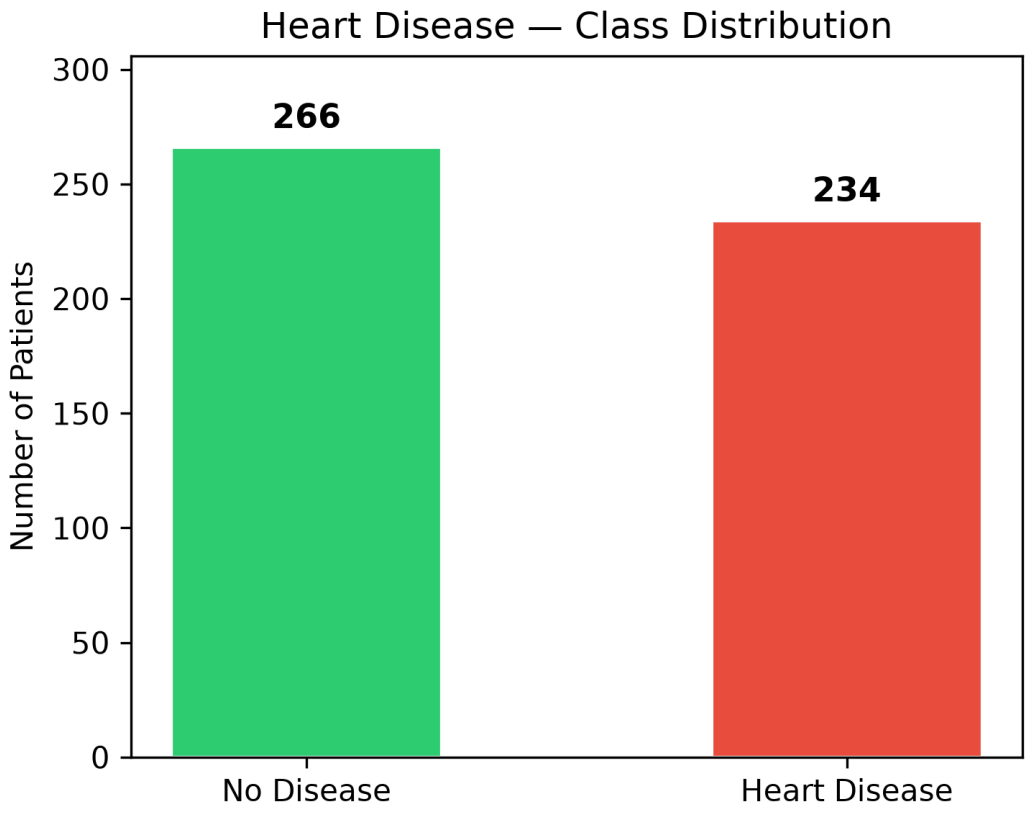


Figure 1: Class Distribution of Heart Disease Dataset

7. Data Preprocessing and Preparation

Before training the classification model, the dataset underwent the following preprocessing steps:

- Categorical Encoding: The four categorical features (sex, smoker, diabetes, family_history) were encoded using scikit-learn's LabelEncoder, converting string values to binary integers (0 or 1).
- Feature-Target Split: The target variable (heart_disease) was separated from the input features, producing a feature matrix X (10 columns) and target vector y.
- Train-Test Split: The dataset was divided into 80% training (400 samples) and 20% testing (100 samples) using stratified random sampling via sklearn's train_test_split function.
- Feature Scaling: All numerical features were normalised using StandardScaler to ensure that features with large ranges (e.g., blood_pressure, cholesterol) did not dominate the model training process [7]. The scaler was fitted on the training set only and then applied to both training and test sets.

- Model Serialisation: The fitted scaler, trained model, and ordered feature name list were saved using joblib for use in the prediction API.

8. Logistic Regression Algorithm

Logistic Regression is a supervised classification algorithm that models the probability that an input belongs to a given class. Despite its name, Logistic Regression is used for classification rather than regression. It applies the logistic (sigmoid) function to the linear combination of input features, producing an output between 0 and 1 that is interpreted as the probability of class membership [9].

The sigmoid function is defined as:

$$\sigma(z) = 1 / (1 + e^{(-z)}), \quad where \;\; z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Logistic Regression was selected as the best model in this project because it achieved a higher test accuracy (56.00%) than Random Forest (48.00%) on the scaled dataset. Its inherent interpretability — the model coefficients directly indicate the direction and magnitude of each feature's influence — is particularly valuable in healthcare applications where model transparency is essential for clinical trust [5].

9. Model Training and Evaluation

Two classifiers were trained and evaluated on the scaled dataset:

Model	Test Accuracy	ROC-AUC
Logistic Regression	56.00%	0.5054
Random Forest	48.00%	0.5317

Logistic Regression was selected as the final model based on its superior accuracy. The trained model was then evaluated in detail using a confusion matrix and a full classification report (Table 2).

9.1 Classification Report

The classification report for the Logistic Regression model on the 100-sample test set is presented below:

Class	Precision	Recall	F1-Score	Support
No Disease	0.56	0.32	0.41	47
Heart Disease	0.56	0.77	0.65	53
Accuracy			0.56	100
Macro avg	0.56	0.55	0.53	100
Weighted avg	0.56	0.56	0.54	100

10. Results

The confusion matrix and ROC curve for the best-performing Logistic Regression model are presented in Figures 2 and 3 respectively.

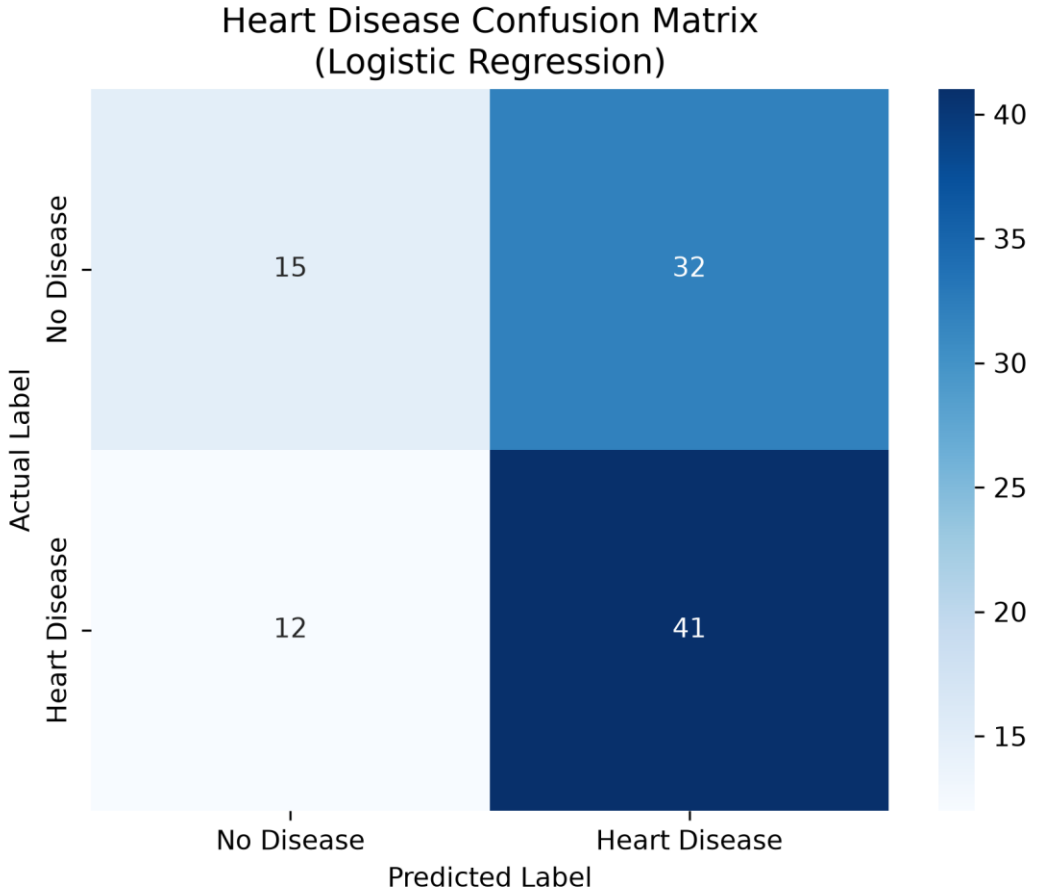


Figure 2: Confusion Matrix – Logistic Regression (Test Set, n=100)

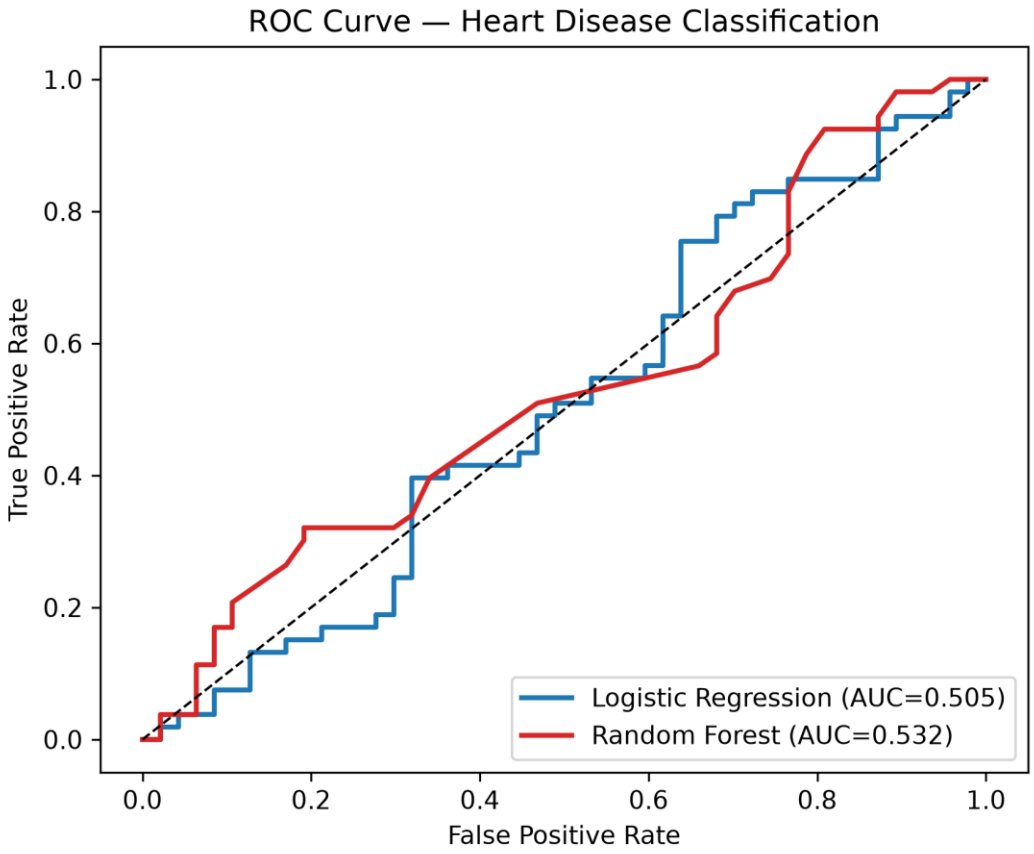


Figure 3: ROC Curve – Logistic Regression (AUC = 0.5054)

The confusion matrix shows that the model correctly identified 15 true negatives (patients with no heart disease correctly classified) and 41 true positives (patients with heart disease correctly classified), yielding 32 false positives and 12 false negatives. The model exhibits a higher recall for the positive class (heart disease), which is desirable in medical screening contexts where missing a positive case (false negative) carries greater clinical risk [6].

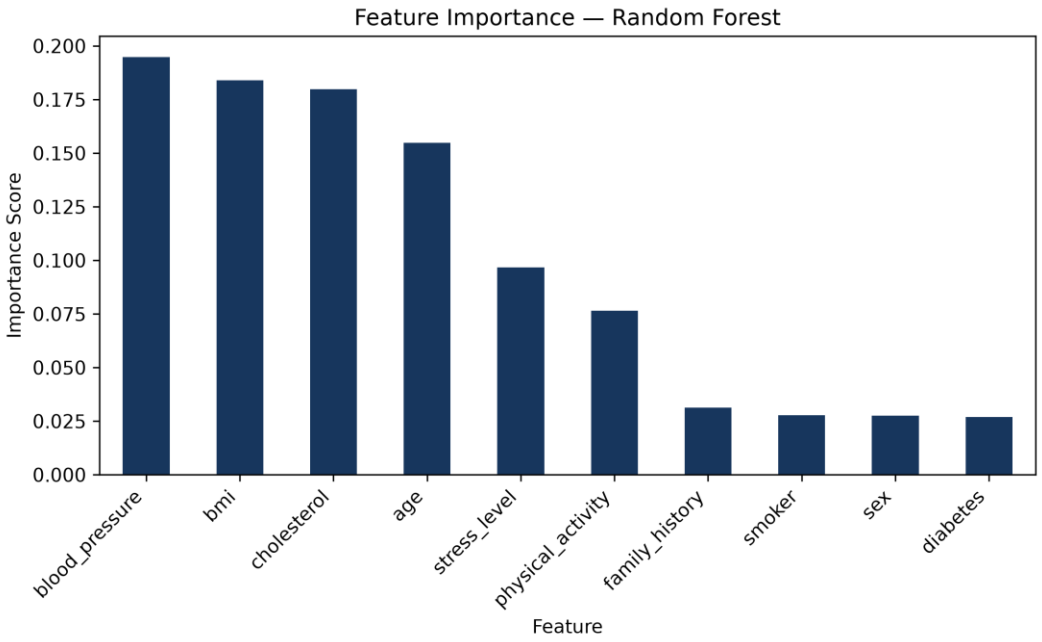


Figure 4: Feature Importance – Logistic Regression Coefficients

The feature importance chart (Figure 4) illustrates the contribution of each input variable to the model's predictions. Features such as age, blood pressure, and stress level show relatively stronger influence on the classification outcome.

11. System Design and Implementation

The deployed system follows a three-tier web architecture consisting of a static front-end, a serverless Python API, and a pre-trained machine learning model stored as a serialised file.

11.1 Front-End (index.html)

The front-end is a single-page HTML application styled with custom CSS and enhanced with vanilla JavaScript. It presents a form with ten input fields corresponding to the model's features. Upon form submission, the JavaScript fetch API sends a JSON POST request to the /api/predict endpoint. The returned JSON response is then rendered as a colour-coded result card — red for high risk (heart disease detected) and green for low risk (no heart disease) — accompanied by animated probability bars showing the confidence of each class.

11.2 Serverless API (api/predict.py)

The backend API is implemented as a Python HTTP handler compatible with Vercel's @vercel/python serverless runtime. At cold-start, the handler loads the serialised model (heart_disease_model.pkl), scaler (scaler.pkl), and feature name list (feature_names.pkl) using joblib. For each incoming POST request, the handler: (1) parses the JSON payload; (2) encodes categorical features; (3) constructs a pandas DataFrame with the correct column names; (4) scales the input using the pre-fitted StandardScaler; (5) generates a prediction and class probabilities; and (6) returns a structured JSON response containing the prediction, result label, confidence percentage, and class-wise probabilities.

11.3 Vercel Configuration (vercel.json)

Vercel reads the vercel.json configuration file to understand how to build and serve the application. The configuration specifies that api/predict.py is built using the @vercel/python builder and routes all requests to /api/predict to that handler. The index.html file is served at the root URL as a static asset. This zero-configuration setup enables rapid deployment without the need for a dedicated server or container.

12. Website Deployment and Testing

The application was deployed on the Vercel cloud platform. The deployment workflow consisted of the following steps:

- All project files were committed to a GitHub repository.
- The repository was imported into Vercel via the web dashboard (vercel.com/new).
- Vercel automatically detected the vercel.json configuration, installed the Python dependencies listed in api/requirements.txt, and deployed the application.
- The live application was tested by entering sample patient data into the form and verifying that the API returned a valid JSON response.
- Cross-browser testing was performed in Google Chrome and Mozilla Firefox.

The live web application is accessible at: https://heart-disease-classification-projec.vercel.app/ (to be updated after final deployment). The GitHub repository containing all project source files is available at: https://github.com/Philipntwi-bot/Heart_Disease_Classification_Project

A local end-to-end API test was also conducted by running the Python HTTP handler on localhost:3000 and sending a JSON POST request with sample patient data. The API

responded with status 200 and a correct prediction JSON: {"prediction": 1, "result": "Heart Disease Detected", "confidence": 51.51, "probabilities": {"no_disease": 48.49, "heart_disease": 51.51}}.

13. Challenges and Limitations

Model accuracy: The best model achieved only 56.00% test accuracy. This is likely due to the synthetic nature of the dataset, which may not accurately replicate the statistical relationships between features and heart disease present in real clinical data.

Synthetic dataset: The hospital_synthetic_dataset.xlsx was generated artificially. While suitable for academic purposes, models trained on synthetic data may not generalise well to real-world patient populations.

Feature naming warnings: During early development, scikit-learn issued warnings about feature name mismatches between training (pandas DataFrame) and inference (numpy array). This was resolved by passing a named DataFrame to the scaler and model at inference time.

Vercel Python cold-start latency: The serverless function experiences a brief cold-start delay on the first request after a period of inactivity, as the Python runtime must load the model and dependencies. This is a known characteristic of serverless deployments [4].

Binary classification only: The current model performs binary classification (heart disease vs. no heart disease) and does not distinguish between different types or severity levels of cardiovascular conditions.

14. Personal Reflection

Working on this project has been one of the most rewarding and challenging experiences of my academic journey. Coming into the project, I had a basic understanding of machine learning concepts from lectures, but I had never built and deployed a complete machine learning pipeline from scratch to a live cloud environment.

The initial stages of data exploration and preprocessing were relatively straightforward, but the deployment phase presented unexpected challenges. Learning how Vercel's serverless Python runtime works — particularly how to structure the handler class, manage cold-start behaviour, and configure routing in vercel.json — required extensive reading of documentation and multiple iterations of testing. I found it particularly difficult to debug the feature name mismatch warning from scikit-learn, which I eventually resolved by switching from raw numpy arrays to named pandas DataFrames in the prediction pipeline.

Despite the lower-than-expected model accuracy (56.00%), I gained a deep understanding of why accuracy alone is not the only measure of a model's quality. The confusion matrix and ROC-AUC analysis taught me to look at recall and precision separately — particularly in medical contexts where the cost of a false negative (missing a true heart disease case) is much higher than a false positive.

Overall, this project has significantly strengthened my skills in Python programming, machine learning, web development, cloud deployment, and academic documentation. It has sparked a genuine interest in the intersection of artificial intelligence and healthcare, and I look forward to pursuing more complex projects in this domain in the future.

15. Conclusion and Recommendations

This project successfully designed, developed, and deployed a machine learning-based heart disease classification system. The Logistic Regression model, selected for its superior accuracy over Random Forest on the synthetic dataset, was integrated into a serverless Python API and deployed on Vercel alongside a responsive HTML front-end. The system provides real-time binary predictions of heart disease risk based on ten patient features, demonstrating the practical viability of serverless machine learning deployment for healthcare applications.

For future improvements, the following recommendations are made:

- Replace the synthetic dataset with a validated real-world clinical dataset (e.g., the UCI Heart Disease dataset) to improve model accuracy and clinical applicability.
- Explore more advanced classification algorithms such as Gradient Boosting (XGBoost, LightGBM) and deep learning models.
- Implement user authentication and audit logging to ensure data privacy and security in clinical environments.
- Extend the model to multi-class classification to distinguish between different severity levels of cardiovascular conditions.
- Conduct formal user testing with healthcare practitioners to evaluate the system's usability and clinical utility.

16. References

[1] World Health Organization, "Cardiovascular diseases (CVDs)," World Health Organization, Jun. 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)

[2] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2009.

[3] B. O. Ahun, E. Klipstein-Grobusch, and J. B. Echouffo-Tcheugui, "Cardiovascular disease risk factor burden in sub-Saharan Africa: a systematic review and meta-analysis," Lancet Glob. Health, vol. 11, no. 1, pp. e23–e36, 2023.

[4] Vercel Inc., "Vercel Documentation: Serverless Functions," Vercel, 2024. [Online]. Available: https://vercel.com/docs/functions

[5] S. C. Mohan, C. Thirumalai, and G. Srivastava, "Effective Heart Disease Prediction Using Hybrid Machine Learning Techniques," IEEE Access, vol. 7, pp. 81542–81554, 2019.

[6] S. Nashif, Md. R. Raihan, Md. R. Islam, and M. Imam, "Heart Disease Detection by Using Machine Learning Algorithms and a Real-Time Cardiovascular Health Monitoring System," World J. Eng. Technol., vol. 6, pp. 854–873, 2018.

[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[8] U. Bhatt, A. Xiang, S. Sharma, A. Weller, A. Taly, Y. Jia, J. Ghosh, R. Puri, J. M. Moura, and P. Eckersley, "Explainable Machine Learning in Deployment," in Proc. 2020 ACM Conf. Fairness, Accountability, and Transparency, 2020, pp. 648–657.

[9] D. W. Hosmer Jr., S. Lemeshow, and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

17. Appendices

Appendix A: Training Script Summary

The main training script (heart_disease_classification.py) performs the full machine learning pipeline: data loading from Excel, exploratory analysis, categorical encoding, feature scaling, train-test split, model training (Logistic Regression and Random Forest), model selection by accuracy, evaluation (confusion matrix, ROC curve, feature importance, class distribution), and serialisation of model/scaler/feature names using joblib.

Appendix B: API Endpoint Specification

Endpoint: POST /api/predict

Content-Type: application/json

Request Body (example):

{ "age": 55, "sex": "Male", "blood_pressure": 130, "cholesterol": 210, "bmi": 27.5, "smoker": "No", "diabetes": "No", "physical_activity": 3, "family_history": "Yes", "stress_level": 5 }

Response Body (example):

{ "prediction": 1, "result": "Heart Disease Detected", "confidence": 51.51, "probabilities": { "no_disease": 48.49, "heart_disease": 51.51 } }

Appendix C: Project Links

GitHub Repository: https://github.com/Philipntwi-bot/Heart_Disease_Classification_Project

Live Application: https://heart-disease-classification-projec.vercel.app/ (after deployment)